PROJECT PHASE -II
PRESENTATION

Truthsight:

Enhancement-Guided
DeepFake Detection in Visual Media

GUIDE:
ANJANA THAMPY S

Members
Ardra Padmakumar (LMC22CS036)
Arsha J S (LMC22CS040)
Arsha P A (LMC22CS041)
Aswathy S (LMC22CS042)

Table of Contents

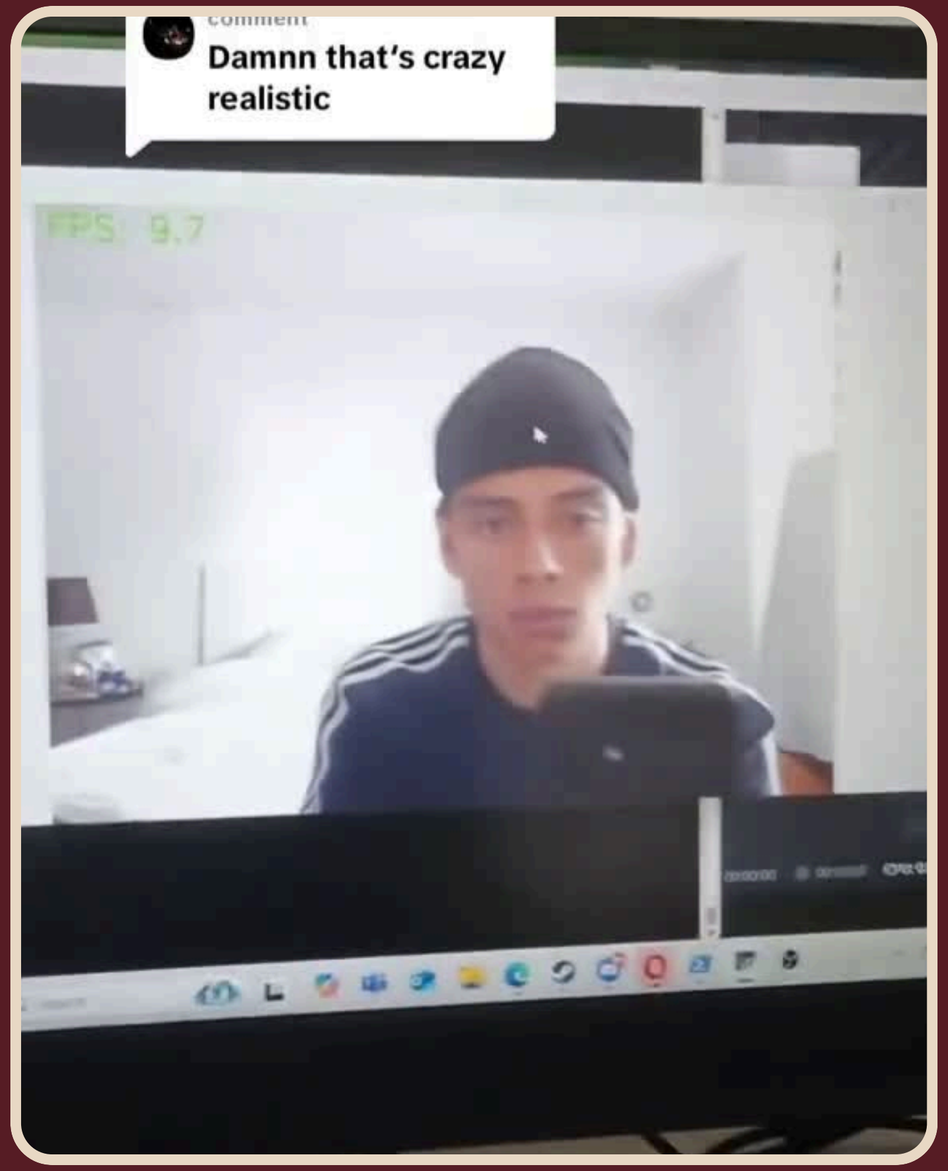
Introduction

TruthSight-Project Introduction

DeepFake technology is becoming increasingly sophisticated, posing serious threats to information integrity, personal security, and public trust.

Detection is especially challenging for low-quality or compressed media, where manipulation artifacts are barely visible.

TruthSight tackles this challenge through a two-stage approach that integrates U-Net–based image enhancement with a ResNet50-based transfer learning classification model, delivering improved detection accuracy and robustness across varied conditions.



Video Credits : @r4f43lo on instagram

DeepFakes of Elon Musk, Central Cee, Robert Downey Jr., and various other celebrities were created and posted by a user on Instagram.

This open-source **Deep-Live-Cam**, currently the number one trending GitHub repository in the world, has taken live deepfaking to the next level—something to be genuinely concerned about.

Literature Review

Sl. No	Author(s) & Year	Method / Approach	Key Findings	Relevance to Project
1	D. M. Akhtar et al., 2023	Attention-Enhanced CNN for multi-dataset deepfake detection	Attention improves feature focus, enabling better detection of subtle forgeries	Supports use of fine-tuned CNN with attention
2	S. Singh et al., 2023	Unified neural framework across images & videos	Achieved adaptability across modalities with shared feature extractors	Inspires architecture design for generalization
3	M. S. H. Mukta et al., 2023	Survey of deepfake models & detection tools	Low-quality and compressed media remain challenging for detectors	Justifies image enhancement and augmentation
4	Guo et al., 2022	Low-light image enhancement with deep learning	Restores detail and contrast without amplifying noise	Validates use of enhancement (UNET + LOL dataset) for better feature extraction

Objective

1. Enhance Visual Data Quality
2. Improve Classification Accuracy
3. Bridge Research Gaps
4. Boost Model Generalization
5. Strengthen Explainability
6. Support Ethical AI Practices
7. Facilitate Easy Integration
8. Continuous Learning & Adaptation
9. Cross-Media Detection
10. User-Friendly Interface

Problem Statement .

Deepfake technology produces highly realistic fake content that is difficult to detect. Existing methods perform poorly on low-quality, compressed, or poorly lit media and often fail to adapt to varied manipulation techniques. A solution is needed that enhances image quality and reliably distinguishes between real and fake content under diverse conditions.

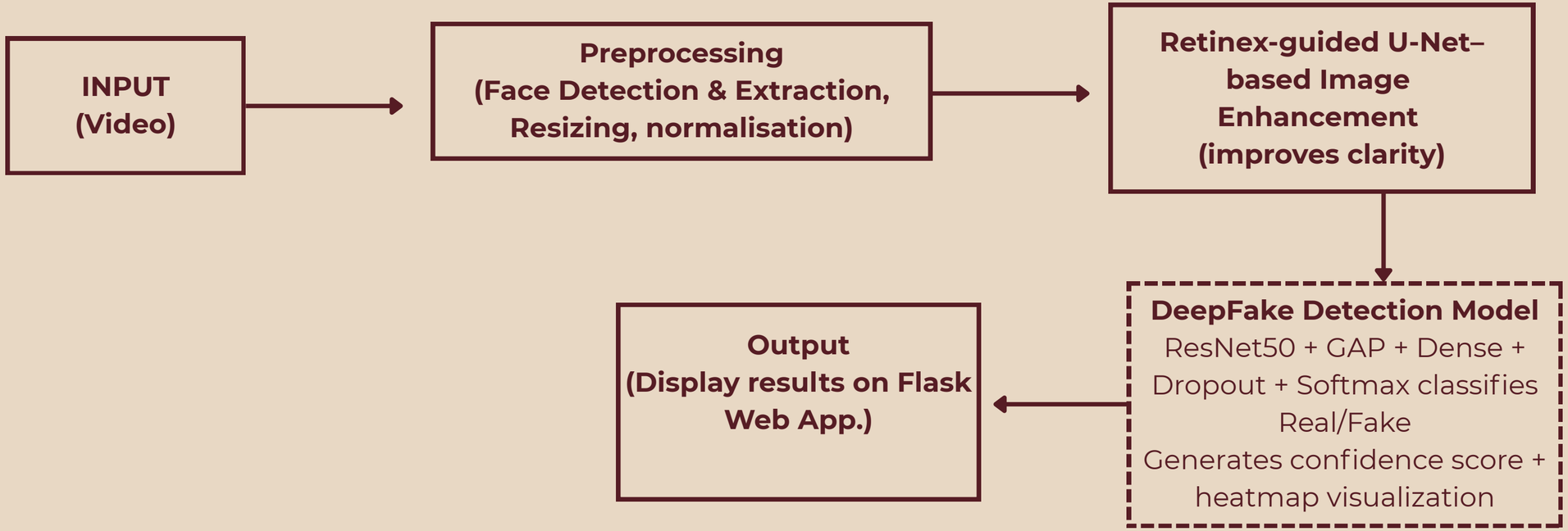
Gap Analysis

Aspect	Previous Approaches	Proposed Approach (TruthSight)
Model Architecture	Single CNN models without enhancement stage.	Dual-stage pipeline: dlib (HOG-based) face detection + Retinex-guided U-Net for image enhancement
Training Method	Mostly trained from scratch, requiring large-scale compute and time.	Transfer learning from pre-trained CNNs; fine-tuning reduces training cost and improves generalization.
Evaluation Metrics	Accuracy-focused, sometimes ignoring perceptual quality.	Uses accuracy, F1, precision, recall, AUC for detection + MSE/SSIM for enhancement evaluation.
Deployment	Focused mainly on model evaluation without deployment or user interaction.	Flask-based web app enabling efficient and user-friendly DeepFake detection.

Proposed Methodology.

- Dataset preparation using DFDC (classification) and LOL (enhancement) datasets
- Frame extraction from videos using OpenCV
- Face detection using dlib (HOG-based model)
- Image enhancement using Retinex-guided U-Net
- DeepFake classification using ResNet50 (transfer learning)
- Majority voting applied for final video-level prediction

Proposed Design.



Software Details	Hardware Requirements
• Language: Python • IDE: VS Code, Jupyter Notebook • Environment: Anaconda • Web Framework: Flask	• Processor: Intel i7 or above • RAM: 16 GB • Storage: 1 TB SSD • GPU: 8 GB VRAM or above

Dataset Description.

Dataset Description

- Source: Kaggle – DeepFake Detection Challenge (DFDC)
- Data Type: .mp4 video files (~10 GB per compressed set)
- Accompanied File: metadata.json with labels and attributes

Metadata Columns:

- filename: Video file name
- label: REAL or FAKE
- original: Source video for manipulated samples

Supporting Files:

- train_sample_videos.zip → Training samples
- test_videos.zip → Evaluation set
- sample_submission.csv → Prediction format

Prediction Goal:

- Classify whether a given video is REAL or FAKE
- Output prediction with confidence score

Data Preprocessing

Prepare high-quality, standardized facial data for the DeepFake detection model.

Face Detection (dlib - HOG Model)

- Detects and crops facial regions from videos.
- Uses HOG-based frontal face detection
- Ensures only relevant facial areas are processed.

Data Augmentation

- Expands dataset & improves model generalization.
- Techniques:
  - Horizontal flip
  - Rotation
  - Brightness/contrast change
  - Shear and zoom transformations

Normalization

- Scales pixel values (0–255 → 0–1).
- Maintains uniform input range and faster convergence.

Algorithm.

dlib HOG Face Detection

- Load frame from video using cv2.VideoCapture()
- Convert frame from BGR to grayscale
- Apply dlib's get_frontal_face_detector() using HOG (Histogram of Oriented Gradients) features
- For each detected face, extract bounding box coordinates (x1, y1, x2, y2)
- Validate face dimensions — skip if width/height < 10px or coordinates out of bounds
- Crop the face region from the enhanced frame
- Resize cropped face to 224×224 for ResNet50 input
- Normalise pixel values to 0–1 range
- Pass to ResNet50 for classification

Image enhancement .

Retinex-Guided U-Net Image enhancement

Purpose

1. *Improve illumination and visual quality of facial images before deepfake detection.*
2. *Make subtle manipulation artifacts more visible under low-light and degraded conditions*

- Applied after face detection to improve facial image quality.
- Retinex theory is used to model illumination variations in low-light images.

Retinex equation used to compute illumination maps:

$$I(x, y) = R(x, y) \times L(x, y)$$

$$\log L \approx \log(I_{high}) - \log(I_{low})$$

- Illumination maps are computed from paired low-light and normal-light images.
- A U-Net model is trained using these Retinex-based illumination maps as supervision.
- Enhances contrast and illumination, making subtle deepfake artifacts more visible.
- Provides clearer and consistent facial inputs for the deepfake detection CNN.

Implementation

- Collected real and fake face images and organized them into separate folders.
- Applied Retinex-guided U-Net–based image enhancement to improve illumination and contrast of facial images.
- Loaded images and resized them to 224 × 224 × 3 to match CNN input requirements.
- Normalized pixel values to the range 0–1 for stable training.
- Assigned labels:
  Real = 1
  Fake = 0
- Converted labels into one-hot encoded format.

Model Training

- Split the dataset into training (75%) and validation (25%) sets.
- Used **ResNet50** pretrained on ImageNet as the feature extractor.
- Removed the top classification layer and added:
  - Global Average Pooling → Dense(256, ReLU) → Dropout(0.5) → Dense(2, Softmax)
- Set base_model.trainable = True (full fine-tuning)
- Compiled the model using Adam optimizer and categorical cross-entropy loss.
- Trained the model with a small batch size and Early Stopping to avoid overfitting.

Working

- Input face images are preprocessed (resizing and normalization)
- Processed images are passed to ResNet50 for feature extraction
- Model learns facial inconsistencies and deepfake artifacts
- Global Average Pooling reduces feature dimensions
- Fully connected layers perform classification
- Softmax layer outputs Real or Fake probabilities
- Early stopping prevents overfitting during training
- Final prediction is generated for the input video

Flask Web Application

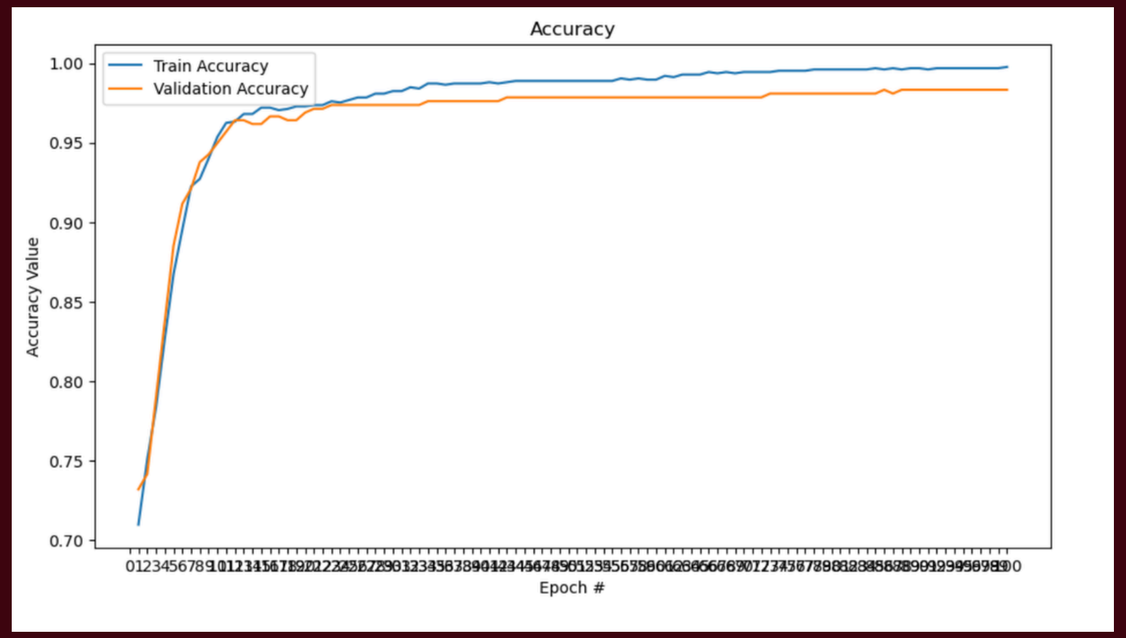
- TruthSight is deployed as a lightweight Flask web application on localhost. It integrates all three models — **dlib (HOG-based) face detector, U-Net enhancement model, and ResNet50** classifier — into a single end-to-end video analysis pipeline.

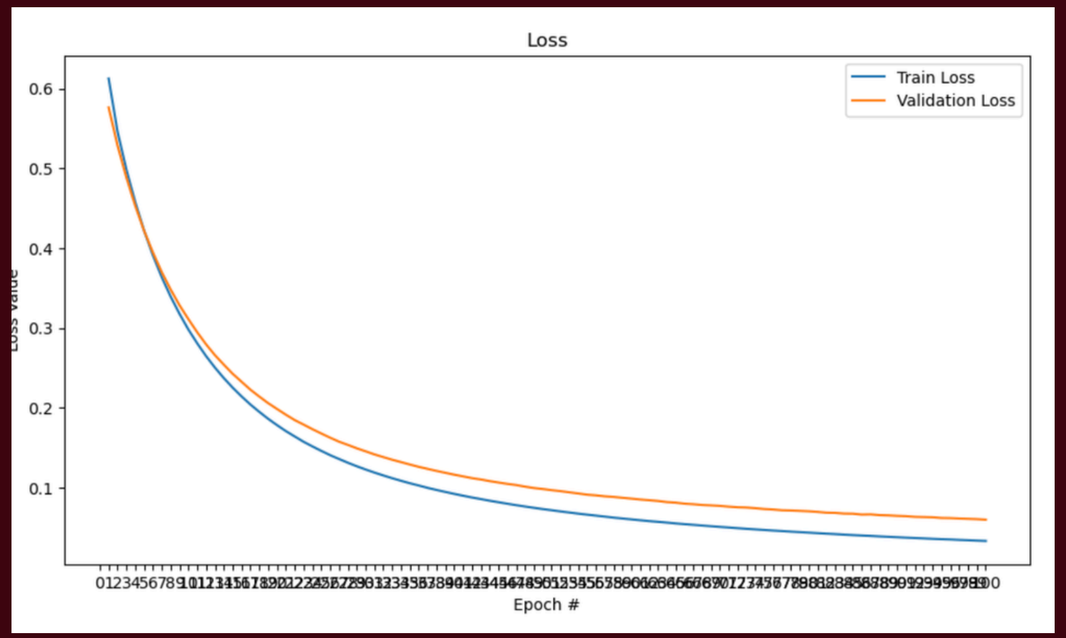
Route	Method	Purpose
/	GET	Renders the video upload form (index.html)
/	POST	Accepts video, processes it, returns result (result.html)
/health	GET	Returns JSON status of all three loaded models

After processing, the app returns a detailed result containing:

- Final label — REAL or FAKE
- Confidence percentage
- Total frames processed
- Number of frames with faces detected
- Video duration in seconds

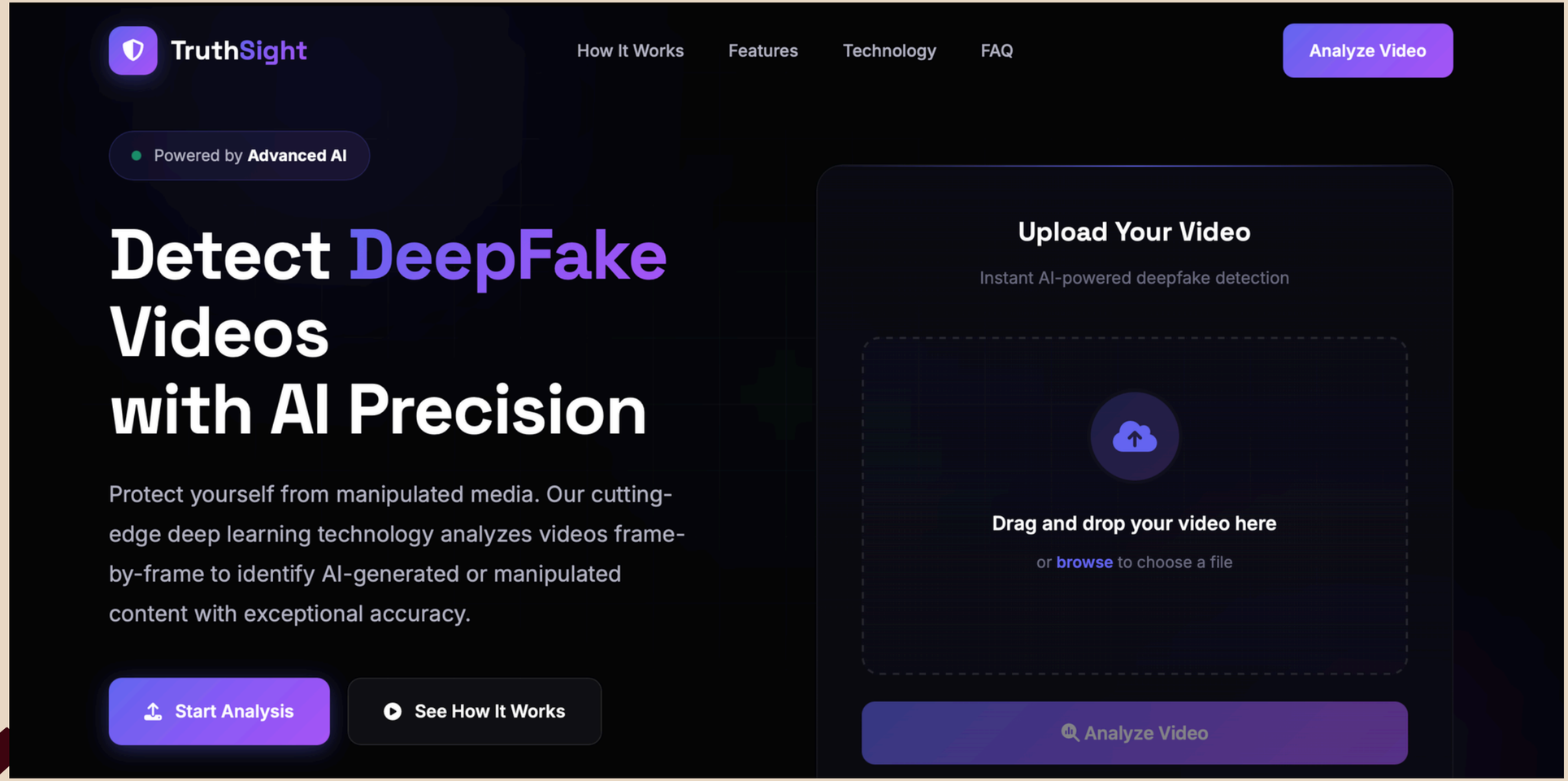
Model Training Performance (ResNet50)





ResNet50 Transfer Learning with Fine-Tuning Performance

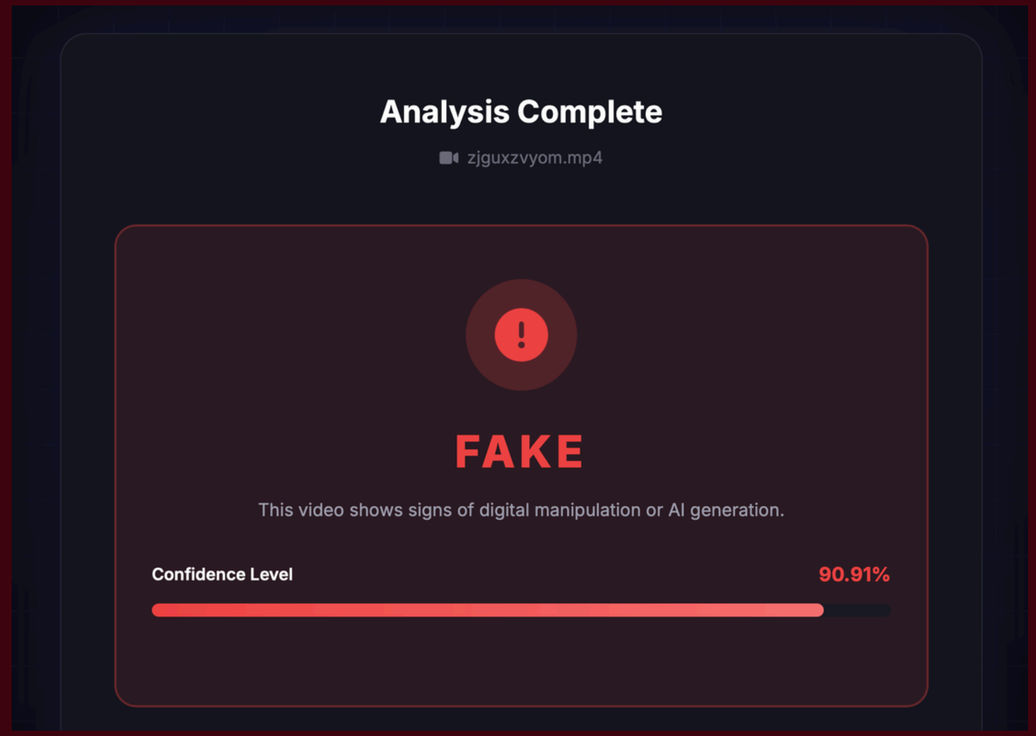
Screenshots



Interface

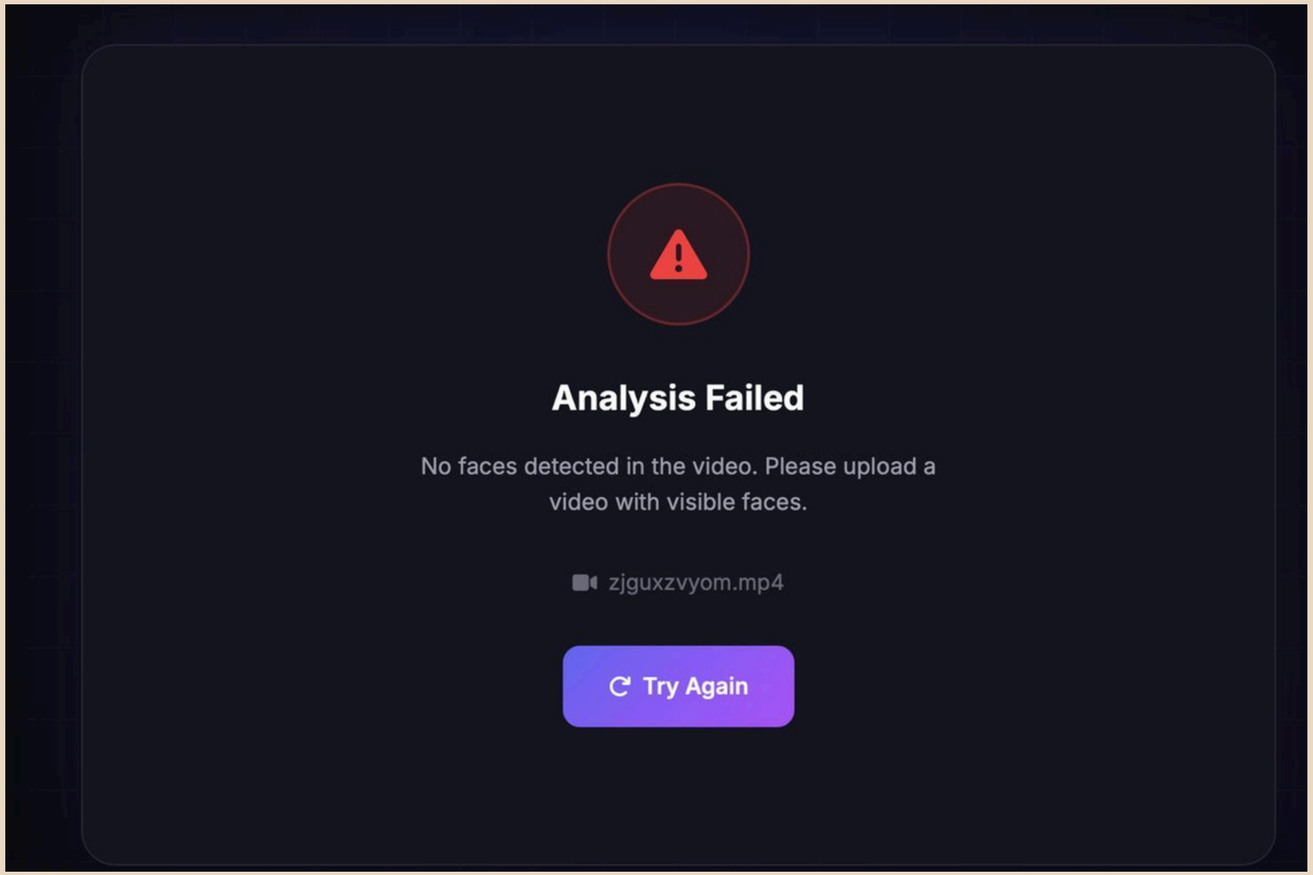
Output

Sample Prediction – Real Video



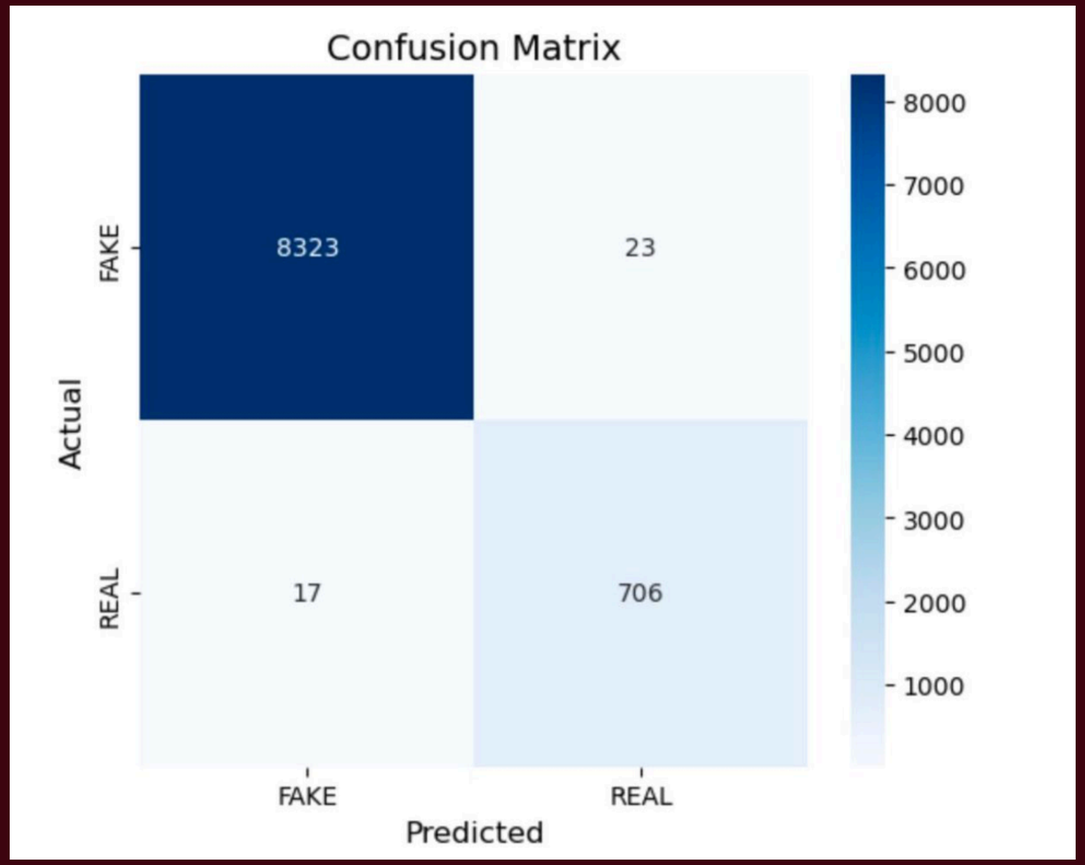
Sample Prediction – Fake Video

Output



Sample Prediction – No Face Detected

Model Evaluation Results




```
Test Accuracy : 99.56%
Precision     : 96.84%
Recall        : 97.65%
F1-score      : 97.25%
```

Conclusion.

TruthSight presents a two-stage DeepFake detection framework that combines Cascade-based face detection, UNet image enhancement, and an attention-augmented CNN classifier to accurately identify manipulated media. By leveraging dataset subsets and pre-trained models with fine-tuning, TruthSight achieves high accuracy while remaining lightweight and scalable. The Flask-based web application provides a user-friendly interface for batch analysis of videos, making the solution practical for real-world multimedia forensics. Overall, TruthSight demonstrates strong generalization, interpretability, and efficiency, offering an effective tool to combat DeepFake content.

Future Scope.

- **Improved preprocessing techniques**

Additional preprocessing such as Error Level Analysis (ELA) or frequency-domain features can be integrated to enhance fake artifact detection.

- **Handling large and diverse datasets**

The system can be trained on larger and more diverse deepfake datasets to improve generalization to unseen fake generation methods.

- **Explainable deepfake detection**

Visualization techniques like Grad-CAM can be added to show which facial regions influenced the model’s decision.

- **Real-time detection support**

With optimization, the same image-based model can be adapted for real-time video streams.

References.

[1] D. M. Akhtar, A. A. Nazib, S. R. Islam, M. A. R. Ahad, and F. A. Afsar, "**Attention-Enhanced CNN for High-Performance Deepfake Detection: A Multi-Dataset Study,**" IEEE Access, vol. 11, pp. 86715–86730, 2023. doi: 10.1109/ACCESS.2023.3283456.

[2] S. Singh, M. Farooq, and M. A. Khan, "**A Unified Neural Framework for Real-Time Deepfake Detection Across Multimedia Modalities to Combat Misleading Content**," IEEE Transactions on Dependable and Secure Computing, early access, pp. 1–14, May 2023. doi: 10.1109/TDSC.2023.3283633.

[3] M. S. H. Mukta, J. Ahmad, M. A. K. Raiaan, S. Islam, S. Azam, M. E. Ali, and M. Jonkman, "**An Investigation of the Effectiveness of Deepfake Models and Tools**," Journal of Sensor and Actuator Networks, vol. 12, no. 4, pp. 1–43, 2023. doi: 10.3390/jsan12040061.

Detecting what the eye cannot see.

Thank you for your time and attention. Looking forward to your valuable feedback and suggestions.

Thank You